

LABORATORY HANDOUT : Generate a Lexical Analyzer Using Lex (or Flex)

1. Shell commands to generate a lexical analyser from a lex script

Write the regular expressions for tokens of interest in the format of lex specifications, which is known as the lex script. Perform the following steps to generate the scanner (also called as lexical analyser).

Step 1 : Run the lex script through lex as shown below.

```
$ lex exp1.l
```

In case there are no errors in the script, lex generates a C program in the file named as “lex.yy.c”. You can read the contents of this generated file now or later by running the ls command.

```
$ ls -l
```

Step 2 : Compile the lex.yy.c program using gcc.

```
$ gcc lex.yy.c -o scan1
```

On successful compilation the executable code is generated by gcc, named as “scan1”, using option “-o” and “scan1” is our executable scanner.

Step 3: For ease of experimentation, you may use relevant features of the shell, such as i/o redirection for using scan1 over the different input files.

```
$ ./scan1 < input1
```

in which case the output of the scanner is displayed on screen. In case you want to save the output of the scanner in a file, use `$ ./scan1 < input1 > output1`

You can easily save the inputs and outputs together in your directory so that your work done during the lab session is recorded for future.

2. File i/o by a Process in Linux

A small digression for those who are NOT familiar with shell programming. In the Unix / Linux OS, a process is attached to three different files by default for communication between the process to the OS. This is realized by the OS by creating a table, named as User Descriptor File Table (UFDT) and linking to every executing process. An entry in UFDT has several information a file, but here we shall only be concerned with the files that an entry points to. The first 3 entries in this table are attached to the standard input device (stdin), the standard output device (stdout), and the standard error device (stderr).

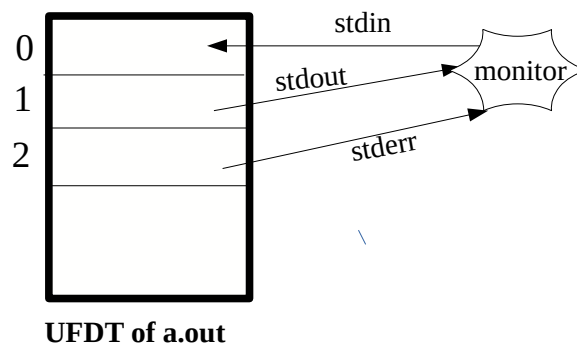


Figure 1 : Default Connections of process to its files for i/o

These connections in the UFDT may be assumed to be done by the shell. As all the 3 device files are mapped to the monitor device by default, the executing process uses this device for input, output and error messages. However an user can instruct the shell to change the first 3 entries in the UFDT (at the indices 0, 1 and 2) by using the shell i/o redirection operators. Consider the following shell command to redirect the UFDT entries of an executing process, say "a.out", as shown below.

```
$ ./a.out < file1 > file2 2> file3
```

The operator "<" refers to the entry at index 0 of UFDT, for redirection of input from the monitor to "file". Similarly, operator ">" or "1>", which refers to the index 1 of UFDT results in redirection of output to "file2", and operator "2>" which refers to index 2 of UFDT redirects the error channel to "file3". The shell changes the UFDT entries as shown below. As a result, execution of "a.out" results in the i/o as desired by the user. The 3 different operators enables an user to change one or all the connections.

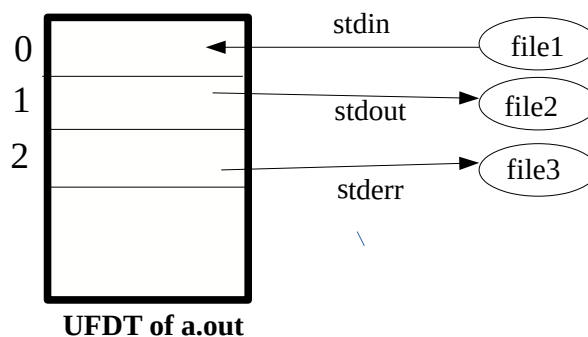


Figure 2 : Modified UFDT Connections of process to its files

3. Summary of shell i/o operators

There are 3 operators used by the shell for input / output redirection (redirecting the standard input, standard output and standard error files respectively, to other files chosen by an user rather than their default binding to device files. Familiarity with these i/o file redirection features of the shell helps in saving our work in files. These help in creating the submission file, to report the work done by a student during a laboratory session.

Operator	Usage	Semantics
<	\$./a.out < file1	The input required by a.out will be read from "file1" when the program reads using scanf() or equivalent functions for reading.
> or 1>	\$./a.out > file2 \$./a.out 1> file2	The output produced by a.out, using printf() or equivalent, will be written to "file2"
2>	\$./a.out 2> file3	The error messages produced by a.out, using the standard error channel, will be written to "file3"
>> or 2>>	\$./a.out >> file4	Same effect as ">" or "2>" except that the data is appended

Operator	Usage	Semantics
	\$./a.out 2>> file5	to the files instead of overwriting. You may also use "1>>" in place of ">>".
1>&2	time (./a.out < input) 2> timeout 1>&2	The parentheses, (./a.out < input), denotes that the input file redirection (<), to read from file "input" applies to the executable "a.out". The argument "2> timeout" specifies that the error channel ("2>") of the "time" command is redirected to the file "timeout". The argument, "1>&2" indicates that the output channel (1>) of time command, which is the output of "a.out", is redirected to the same file as that of "2>", that is the file, "timeout". The effect of the entire shell command is that the file "timeout" contains the output of "a.out" followed by the output of "time".
2>&1	time (./a.out < input) 1> timeout 2>&1	The explanation is similar to "1>&2" and the effect is the same, even though the order of i/o redirections are reversed.
1>&2	time (./scan1 < input3) 2>> timeout 1>&2	The effect is similar to "1>&2", except that in this case the the argument "2>>" indicates that the output of "time"

End of Document